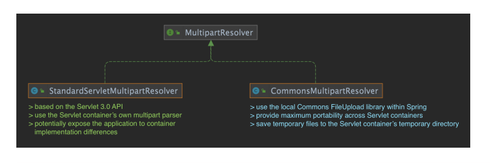
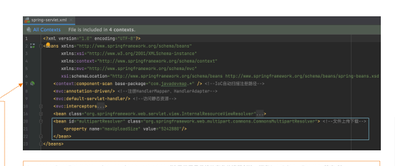


StandardServletMultipartResolver
> based on the Servlet 3.0 API
> use the Servlet container's own multipart parser
> potentially expose the application to container implementation differences

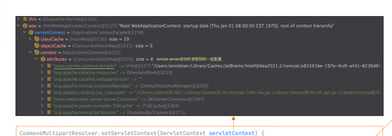
CommonsMultipartResolver
> use the local Commons FileUpload library within Spring
> provide maximum portability across Servlet containers
> save temporary files to the Servlet container's temporary directory

```
1. 启动tomcat server
  > 读取解析server.xml → start a StandardServer → Service
    > 启动Engine: Host -> Context -> Wrapper
    > 启动Connector

2. 部署war包
  > 创建StandardContext context
  > 解压war包
  > 解析web.xml, 将配置内容设置到context中
  > host.addChild(context), 触发context.start()
    > findChildren: 查找所有子容器Wrappers
    > loandOnStartUp: 遍历所有Wrappers, 执行Wrapper.load()
      > loadeServlet: 创建DispatcherServlet实例servlet, 执行servlet.init()
        > 创建XmlWebApplicationContext
          > loadBeanDefinitions: 获取web.xml中contextConfigLocation的所有xml文件, 读取内容, 生成beanDefinitions, 注册到beanFactory
          > 注册non-lazy-init singletons: 创建CommonsMultipartResolver bean
        > 初始化spring mvc九大组件
  > 监控war包, 有变化时重新部署
```



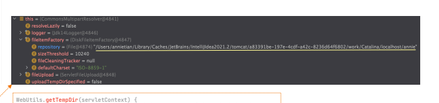
```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
    <context:component-scan base-package="com.javadevmap.*" /> <!--IoC自动扫描注册路径-->
    <mvc:annotation-driven/> <!--注册HandlerMapper、HandlerAdapter-->
    <mvc:default-servlet-handler/> <!--访问静态资源-->
    <mvc:interceptors...>
    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver"...>
    <bean id="multipartResolver" class="org.springframework.web.multipart.commons.CommonsMultipartResolver"> <!--文件上传下载-->
        <property name="maxUploadSize" value="5242880"/>
    </bean>
</beans>
```

```
1. createBeanInstance: instantiate the bean (这里使用最传统的有参构造器创建, 还有Supplier、Factory等方式)
  > resolveBeanClass: 加载类
  > resolveConstructorArguments: 解析参数beanDefinition中的构造器参数args
  > 匹配合适的构造器ctor
  > 反射调用ctor.newInstance(args)
2. applyMergedBeanDefinitionPostProcessors: allow post-processors to modify the merged bean definition
3. populateBean: 注入依赖
4. initializeBean
  > applyBeanPostProcessorsBeforeInitialization()
    > ServletContextAwareProcessor.postProcessBeforeInitialization()
      > bean.setServletContext()
5. registerDisposableBeanIfNecessary
```



```
> this = {DispatcherServlet@3126}
v wac = {XmlWebApplicationContext@3125} "Root WebApplicationContext: startup date [Thu Jan 01 08:00:00 CST 1970]; root of context hierarchy"
  v servletContext = {ApplicationContextFacade@3158}
    > classCache = {HashMap@3230}  size = 29
      objectCache = {ConcurrentHashMap@3231}  size = 0
    v context = {ApplicationContext@3232}
      v attributes = {ConcurrentHashMap@3233}  size = 8  tomcat server启动时读取到的一些配置
        > "javax.servlet.context.tempdir" -> {File@3257} "/Users/annietian/Library/Caches/JetBrains/IntelliJIdea2021.2/tomcat/a83391be-197e-4cdf-a42c-8236d6
        > "org.apache.catalina.resources" -> {StandardRoot@3259}
        > "org.apache.catalina.webappVersion" -> ""
        > "org.apache.tomcat.InstanceManager" -> {DefaultInstanceManager@3263}
        > "org.apache.catalina.jsp_classpath" -> "/Library/tomcat8/lib/:/Library/tomcat8/lib/tomcat-i18n-ko.jar:/Library/tomcat8/lib/el-api.jar:/Library/tomcat8/li
        > "javax.websocket.server.ServerContainer" -> {WsServerContainer@3267}
        > "org.apache.jasper.compiler.TldCache" -> {TldCache@3269}
        > "org.apache.tomcat.JarScanner" -> {StandardJarScanner@3271}
```

```
CommonsMultipartResolver.setServletContext(ServletContext servletContext) {
  if (!isUploadTempDirSpecified()) getFileItemFactory().setRepository(WebUtils.getTempDir(servletContext));
}
```



```
v this = {CommonsMultipartResolver@4841}
    resolveLazily = false
  > logger = {Jdk14Logger@4846}
  v fileItemFactory = {DiskFileItemFactory@4847}
      repository = {File@4874} "/Users/annietian/Library/Caches/JetBrains/IntelliJIdea2021.2/tomcat/a83391be-197e-4cdf-a42c-8236d64f6802/work/Catalina/localhost/annie"
      sizeThreshold = 10240
      fileCleaningTracker = null
    > defaultCharset = "ISO-8859-1"
  > fileUpload = {ServletFileUpload@4848}
    uploadTempDirSpecified = false
```

```
WebUtils.getTempDir(servletContext) {
  return (File) servletContext.getAttribute("javax.servlet.context.tempdir");
  //javax.servlet.context.tempdir在tomcat启动时初始化
}
```

3. 浏览器发送请求

浏览器

1.请求文件上传页面 http://localhost:8081/annie/test1/file/uploadFile

2.填写form表单，上传文件 http://localhost:8081/annie/test1/file/upload

3.分析返回的status页面，发现引用其他图片 <img src="download?filename=javadevmap.png">
发送子请求获取图片 http://localhost:8081/annie/test1/file/download?filename=javadevmap.png

4.所有资源都拿到了，渲染最终页面

服务器

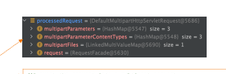
1.返回upload页面

2.处理文件，返回status页面
```
  1).checkMultipart: 检查request类型，发现是multipart/*, 将其转成multipart request
     > 以header中的boundary为界限，拆分并解析body中的stream
       > 常规类型的参数: 将stream中的name和value转成String, 存入multipartParameters
       > 文件类型的参数: 小于10KB的，缓存在内存里，超过时存入磁盘的临时文件。将文件信息存入multipartFiles
     > 将解析出来的请求参数封入multipart request
  2).getHandler: 根据请求路径匹配Controller handler method和适用的拦截器，组成执行链
  3).getHandlerAdapter: 根据handler类型，选择对应的适配器ha
  4).ha执行handler
     > FileUploadController.uploadProductPic()
     > 处理返回值: 将返回的view name设置到mavContainer中
  5).根据返回的view name和model数据，解析并渲染视图，存入response.outputStream.charBuffer
  6).tomcat connector从response.outputStream.charBuffer中获取html内容，生成http响应报文
```

3.返回指定文件
```
  1).Controller的handler method查找文件并返回
  2).Spring将返回的文件写入response.outputStream.byteBuffer
  3).tomcat connector从response.outputStream.byteBuffer获取文件内容，生成http响应报文
```

```
//Fiddler抓到的请求包
POST http://localhost:8081/annie/test1/file/upload HTTP/1.1
Host: localhost:8081
Connection: keep-alive
Content-Length: 427174
Cache-Control: max-age=0
sec-ch-ua: "Not?A_Brand";v="8", "Chromium";v="108", "Google Chrome";v="108"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "macOS"
Upgrade-Insecure-Requests: 1
Origin: http://localhost:8081
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryK1uhP22QYOW8PZeX //分割表单多个字段
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/108.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://localhost:8081/annie/test1/file/uploadFile
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7
Cookie: JSESSIONID=C99046ADA2CA4698BBEEAF50FF259466

------WebKitFormBoundaryK1uhP22QYOW8PZeX
Content-Disposition: form-data; name="productId"

1
------WebKitFormBoundaryK1uhP22QYOW8PZeX
Content-Disposition: form-data; name="description"

嘿嘿嘿嘿嘿
------WebKitFormBoundaryK1uhP22QYOW8PZeX
Content-Disposition: form-data; name="file"; filename="apple.png"
Content-Type: image/png
********************************

------WebKitFormBoundaryK1uhP22QYOW8PZeX
Content-Disposition: form-data; name="上传"

Submit
------WebKitFormBoundaryK1uhP22QYOW8PZeX--
```

```
… → void DispatcherServlet.doDispatch(HttpServletRequest request, HttpServletResponse response) throws Exception{
  HttpServletRequest processedRequest = request;
  HandlerExecutionChain mappedHandler = null;
  boolean multipartRequestParsed = false;
  WebAsyncManager asyncManager = WebAsyncUtils.getAsyncManager(request);
  try {
    ModelAndView mv = null; Exception dispatchException = null;
    try {
      //1.解析request body中的form data stream, 构造multipart request
      processedRequest = checkMultipart(request);
      mappedHandler = getHandler(processedRequest);
      if (mappedHandler == null) { noHandlerFound(processedRequest, response); return; }
      HandlerAdapter ha = getHandlerAdapter(mappedHandler.getHandler());
      if (!mappedHandler.applyPreHandle(processedRequest, response))  return;
      mv = ha.handle(processedRequest, response, mappedHandler.getHandler()); //2.交给Controller处理请求
      applyDefaultViewName(processedRequest, mv);
      mappedHandler.applyPostHandle(processedRequest, response, mv);
    }
    catch (Exception ex) dispatchException = ex;
    catch (Throwable err) dispatchException = new ServletException("Handler dispatch failed: " + err, err);
    /* spring解析视图, 并将model中的数据一起交给tomcat JspServlet渲染html页面
     * tomcat JspServlet从磁盘载入jsp文件 > 转成java > 编译 > 创建实例 > 调用_jspService()生成html
     * > 最后存入request.response.outputStream.charBuffer  */
    processDispatchResult(processedRequest, response, mappedHandler, mv, dispatchException);
  }
  catch(Exception ex) triggerAfterCompletion(processedRequest, response, mappedHandler, ex);
  fianlly {…}
}
```



```
v processedRequest = {DefaultMultipartHttpServletRequest@5686}
  > multipartParameters = {HashMap@5547}  size = 3
  > multipartParameterContentTypes = {HashMap@5548}  size = 3
  > multipartFiles = {LinkedMultiValueMap@5690}  size = 1
  > request = {RequestFacade@5630}
```

```
//Convert the request into a multipart request
HttpServletRequest checkMultipart(HttpServletRequest request) {
  if (this.multipartResolver != null && this.multipartResolver.isMultipart(request)) {
    return this.multipartResolver.resolveMultipart(request);
  }
  return request;
}
```

```
//CASE1: 使用StandardServletMultipartResolver
StandardServletMultipartResolver.resolveMultipart() …→ request.getParts()

org.apache.catalina.connector.Request.parseParameters() {
  String contentType = getContentType();
  if ("multipart/form-data".equals(contentType)) {
    parseParts(false);
    success = true;
    return;
  }
}

parseParts(boolean explicit) {
  MultipartConfigElement mce = getWrapper().getMultipartConfigElement(); //配置信息
  DiskFileItemFactory factory = new DiskFileItemFactory(); //创建FileItemFactory对象
  factory.setRepository(location.getCanonicalFile());
  factory.setSizeThreshold(mce.getFileSizeThreshold());
  ServletFileUpload upload = new ServletFileUpload();
  upload.setFileItemFactory(factory);
  upload.setFileSizeMax(mce.getMaxFileSize());
  upload.setSizeMax(mce.getMaxRequestSize());
  parts = new ArrayList<>();
  //1.解析request body中的files
  List<FileItem> items = upload.parseRequest(new ServletRequestContext(this));
  //2.将files封装成ApplicationParts, 存入Request.parts中
  for (FileItem item : items) {
    ApplicationPart part = new ApplicationPart(item, location);
    parts.add(part);
  }
}
```

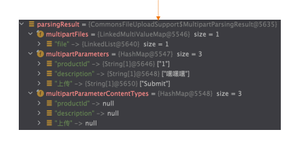
```
List<FileItem> ServletFileUpload.parseRequest() {
  List<FileItem> items = new ArrayList<>();
  FileItemIterator iter = getItemIterator(ctx);
  FileItemFactory fac = getFileItemFactory();
  while (iter.hasNext()) { //遍历迭代器, 创建一个个FileItem对象
    final FileItemStream item = iter.next();
    final String fileName = ((FileItemIteratorImpl.FileItemStreamImpl) item).name;
    FileItem fileItem = fac.createItem(item.getFieldName(), item.getContentType(), item.isFormField(), fileName);
    items.add(fileItem);
    //读取request body中的文件流, 存入临时文件中
    Streams.copy(item.openStream(), fileItem.getOutputStream(), true);
    final FileItemHeaders fih = item.getHeaders();
    fileItem.setHeaders(fih);
  }
  return items;
}
```

```
… → FileItemIteratorImpl(RequestContext ctx) {
  String contentType = ctx.getContentType(); //multipart/form-data; boundary=----WebKitFormBoundaryCleiBkxYybUVQAGq
  final int contentLengthInt = ctx.getContentLength();
  final long requestSize = UploadContext.class.isAssignableFrom(ctx.getClass()) ? ((UploadContext) ctx).contentLength()
                                                                               : contentLengthInt;
  InputStream input;
  if (sizeMax >= 0) {
    input = new LimitedInputStream(ctx.getInputStream(), sizeMax) {…}
  } else input = ctx.getInputStream();
  String charEncoding = headerEncoding; //UTF-8
  boundary = getBoundary(contentType);
  notifier = new MultipartStream.ProgressNotifier(listener, requestSize);
  multi = new MultipartStream(input, boundary, notifier);
  multi.setHeaderEncoding(charEncoding);
  skipPreamble = true;
  findNextItem();
}
```

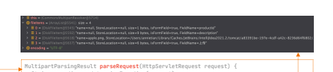
```
MultipartStream(InputStream input, byte[] boundary, int bufSize, MultipartStream.ProgressNotifier pNotifier) {
  this.boundaryLength = boundary.length + BOUNDARY_PREFIX.length;
  this.input = input;
  this.bufSize = Math.max(bufSize, this.boundaryLength * 2);
  this.buffer = new byte[this.bufSize];
  this.notifier = pNotifier;
  this.boundary = new byte[this.boundaryLength];
  this.boundaryTable = new int[this.boundaryLength + 1];
  this.keepRegion = this.boundary.length;
  System.arraycopy(BOUNDARY_PREFIX, 0, this.boundary, 0, BOUNDARY_PREFIX.length);
  System.arraycopy(boundary, 0, this.boundary, BOUNDARY_PREFIX.length, boundary.length);
  this.computeBoundaryTable();
  this.head = 0;
  this.tail = 0;
}
```

```
boolean findNextItem() throws IOException {
  if (eof) return false; //stream读完了, 返回
  if (currentItem != null) {…} //清空当前item
  for (;;)
    boolean nextPart;
    if (skipPreamble) nextPart = multi.skipPreamble(); //true
    else {…}
    if (!nextPart) {…}
    FileItemHeaders headers = getParsedHeaders(multi.readHeaders());
    if (currentFieldName == null) {
      String fieldName = getFieldName(headers); //productId
      if (fieldName != null) {
        String subContentType = headers.getHeader(CONTENT_TYPE);
        if (subContentType != null && subContentType.toLowerCase(Locale.ENGLISH).startsWith(MULTIPART_MIXED)) {…}
        String fileName = getFileName(headers); //fileName为空的是form field
        currentItem = new FileItemStreamImpl(fileName,fieldName, headers.getHeader(CONTENT_TYPE),fileName == null, getContentLength(headers));
        currentItem.setHeaders(headers);
        notifier.noteItem();
        itemValid = true;
        return true;
      }
    }
    else {…}
    multi.discardBodyData();
  }
}
```

```
//CASE2: 使用CommonsMultipartResolver
MultipartHttpServletRequest CommonsMultipartResolver.resolveMultipart(final HttpServletRequest request) {
  //1.解析request中的stream
  MultipartParsingResult parsingResult = parseRequest(request);
  //2.将解析结果构造成DefaultMultipartHttpServletRequest
  return new DefaultMultipartHttpServletRequest(request, parsingResult.getMultipartFiles(),
                                               parsingResult.getMultipartParameters(),
                                               parsingResult.getMultipartParameterContentTypes());
}
```



```
v parsingResult = {CommonsFileUploadSupport$MultipartParsingResult@5635}
  v multipartFiles = {LinkedMultiValueMap@5546}  size = 1
    > "file" -> {LinkedList@5640}  size = 1
  v multipartParameters = {HashMap@5547}  size = 3
    > "productId" -> {String[1]@5646} ["1"]
    > "description" -> {String[1]@5648} ["嘿嘿嘿"]
    > "上传" -> {String[1]@5650} ["Submit"]
  v multipartParameterContentTypes = {HashMap@5548}  size = 3
    > "productId" -> null
    > "description" -> null
    > "上传" -> null
```



```
> this = {CommonsMultipartResolver@5714}
v fileItems = {ArrayList@5541}  size = 4
  > 0 = {DiskFileItem@5545} "name=null, StoreLocation=null, size=1 bytes, isFormField=true, FieldName=productId"
  > 1 = {DiskFileItem@5592} "name=null, StoreLocation=null, size=9 bytes, isFormField=true, FieldName=description"
  > 2 = {DiskFileItem@5618} "name=apple.png, StoreLocation=/Users/annietian/Library/Caches/JetBrains/IntelliJIdea2021.2/tomcat/a83391be-197e-4cdf-a42c-8236d64f6802/
  > 3 = {DiskFileItem@5637} "name=null, StoreLocation=null, size=6 bytes, isFormField=true, FieldName=上传"
> encoding = "UTF-8"
```

```
MultipartParsingResult parseRequest(HttpServletRequest request) {
  String encoding = determineEncoding(request);
  FileUpload fileUpload = prepareFileUpload(encoding);
  //1.拆分request中的form data stream, file类型的暂存到临时文件中
  List<FileItem> fileItems = ((ServletFileUpload)fileUpload).parseRequest(request);
  //2.提取fileItems中的信息, 分类整理
  return parseFileItems(fileItems, encoding);
}
```

```
MultipartParsingResult parseFileItems(List<FileItem> fileItems, String encoding) {
  MultiValueMap<String, MultipartFile> multipartFiles = new LinkedMultiValueMap<>(); //暂存文件
  Map<String, String[]> multipartParameters = new HashMap<>(); //暂存常规参数
  Map<String, String> multipartParameterContentTypes = new HashMap<>();
  for (FileItem fileItem : fileItems) {
    if (fileItem.isFormField()) { //CASE1:常规表单参数
      String value;
      String partEncoding = determineEncoding(fileItem.getContentType(), encoding);
      value = fileItem.getString(partEncoding); //stream → String
      String[] curParam = multipartParameters.get(fileItem.getFieldName());
      if (curParam == null) multipartParameters.put(fileItem.getFieldName(), new String[] {value});
      else {
        String[] newParam = StringUtils.addStringToArray(curParam, value);
        multipartParameters.put(fileItem.getFieldName(), newParam);
      }
      multipartParameterContentTypes.put(fileItem.getFieldName(), fileItem.getContentType());
    }
    else { //CASE2: 文件
      CommonsMultipartFile file = createMultipartFile(fileItem);
      multipartFiles.add(file.getName(), file);
    }
  }
  return new MultipartParsingResult(multipartFiles, multipartParameters, multipartParameterContentTypes);
}
```

```
List<FileItem> FileUploadBase.parseRequest(RequestContext ctx) {
  List<FileItem> items = new ArrayList(); boolean successful = false;
  FileItemIterator iter = getItemIterator(ctx);
  FileItemFactory fac = getFileItemFactory();
  try {
    while (iter.hasNext()) {
      final FileItemStream item = iter.next();
      final String fileName = ((FileItemIteratorImpl.FileItemStreamImpl) item).name;
      FileItem fileItem = fac.createItem(item.getFieldName(), item.getContentType(),item.isFormField(), fileName);
      items.add(fileItem);
      //item.openStream():
      //fileItem.getOutputStream(): 创建temp file > 构造DeferredFileOutputStream
      Streams.copy(item.openStream(), fileItem.getOutputStream(), true);
      final FileItemHeaders fih = item.getHeaders();
      fileItem.setHeaders(fih);
    }
    successful = true;
    return items;
  } catch(…) {…}
  finally {
    if (!successful) for (FileItem fileItem : items) fileItem.delete();
  }
}
```

```
OutputStream DiskFileItem.getOutputStream() throws IOException {
  if (this.dfos == null) {
    File outputFile = getTempFile(); //创建临时文件
    this.dfos = new DeferredFileOutputStream(this.sizeThreshold, outputFile);
  }
  return this.dfos;
}

File getTempFile() {
  if (this.tempFile == null) {
    File tempDir = this.repository; //tomcat临时文件存放目录
    if (tempDir == null) tempDir = new File(System.getProperty("java.io.tmpdir"));
    String tempFileName = String.format("upload_%s_%s.tmp", UID, getUniqueId());
    this.tempFile = new File(tempDir, tempFileName);
  }
  return this.tempFile;
}
```

```
DeferredFileOutputStream(int threshold, File outputFile, String prefix, String suffix, File directory) {
    super(threshold);
    this.closed = false;
    this.outputFile = outputFile;
    this.memoryOutputStream = new ByteArrayOutputStream();
    this.currentOutputStream = this.memoryOutputStream;
    this.prefix = prefix;
    this.suffix = suffix;
    this.directory = directory;
}
```

```
long Streams.copy(InputStream inputStream, OutputStream outputStream, boolean closeOutputStream) {
  return copy(inputStream, outputStream, closeOutputStream, new byte[8192]);
}

long copy(InputStream inputStream, OutputStream outputStream, boolean closeOutputStream, byte[] buffer) {
  OutputStream out = outputStream;
  InputStream in = inputStream;
  try {
    long total = 0L;
    while(true) {
      int res = in.read(buffer); //1.读取file stream
      if (res > 0) {
        total += (long)res;
        if (out != null) out.write(buffer, 0, res); //2.写入DeferredFileOutputStream.currentOutputStream
      }
      if (res == -1) {
        if (out != null) {
          if (closeOutputStream) out.close();
          else out.flush();
          out = null;
        }
        in.close(); in = null;
        return total
      }
    }
  } finally {
    IOUtils.closeQuietly(in);
    if (closeOutputStream) IOUtils.closeQuietly(out);
  }
}
```

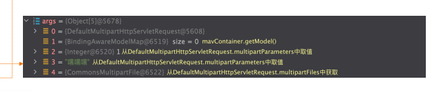
```
/* 1. 小于阈值10KB时, 存内存
 * 2. 达到缓存阈值后, 转存磁盘: 将缓存数据刷入文件, 后续数据直接写入文件 */
public void ThresholdingOutputStream.write(byte[] b, int off, int len) {
    this.checkThreshold(len);
    //ByteArrayOutputStream or FileOutputStream
    this.getStream().write(b, off, len);
    this.written += (long)len;
}
```

```
//达到缓存阈值了, 更换output stream
… → void DeferredFileOutputStream.thresholdReached() {
  FileOutputStream fos = new FileOutputStream(this.outputFile);
  this.memoryOutputStream.writeTo(fos); //已缓存的数据写入temp file
  this.currentOutputStream = fos; //后续数据直接写入文件
  this.memoryOutputStream = null;
}
```

```
RequestMappingHandlerAdapter构造
→ void ServletInvocableHandlerMethod.invokeAndHandle(ServletWebRequest webRequest, ModelAndViewContainer mavContainer, Object... providedArgs) {
  Object returnValue = invokeForRequest(webRequest, mavContainer, providedArgs); //业务逻辑处理请求
  setResponseStatus(webRequest);
  if (returnValue == null) {…} else if (StringUtils.hasText(getResponseStatusReason())) {…}
  mavContainer.setRequestHandled(false);
  this.returnValueHandlers.handleReturnValue(returnValue, getReturnValueType(returnValue), mavContainer, webRequest); //处理返回值
}
```

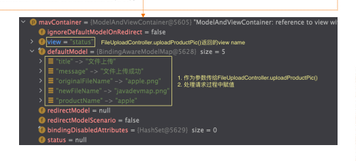
```
Object invokeForRequest(NativeWebRequest request, ModelAndViewContainer mavContainer, Object... providedArgs) {
  Object[] args = getMethodArgumentValues(request, mavContainer, providedArgs); //准备方法入参
  return doInvoke(args);
}

Object doInvoke(Object... args) {
  Method method = getBridgedMethod();
  try {
    if (KotlinDetector.isSuspendingFunction(method)) {…}
    return method.invoke(getBean(), args); //调用FileUploadController实例的uploadProductPic()
  }
  catch (…) {…}
}
```

```
Object[] getMethodArgumentValues(NativeWebRequest request, ModelAndViewContainer mavContainer, Object... providedArgs) {
  MethodParameter[] parameters = getMethodParameters(); //获取方法声明中的参数列表
  if (ObjectUtils.isEmpty(parameters)) return EMPTY_ARGS; //方法没有参数, 直接返回
  Object[] args = new Object[parameters.length];
  for (int i = 0; i < parameters.length; i++) { //遍历所有参数, 使用对应的解析器从request中获取参数值
    MethodParameter parameter = parameters[i];
    parameter.initParameterNameDiscovery(this.parameterNameDiscoverer);
    args[i] = findProvidedArgument(parameter, providedArgs);
    if (args[i] != null) continue;
    args[i] = this.resolvers.resolveArgument(parameter, mavContainer, request, this.dataBinderFactory);
  }
  return args; //返回组装好的参数列表
}
```



```
v args = {Object[5]@5678}
  > 0 = {DefaultMultipartHttpServletRequest@5608}
    1 = {BindingAwareModelMap@6519}  size = 0  mavContainer.getModel()
  > 2 = {Integer@6520} 1 从DefaultMultipartHttpServletRequest.multipartParameters中取值
  > 3 = "嘿嘿嘿" 从DefaultMultipartHttpServletRequest.multipartParameters中取值
  > 4 = {CommonsMultipartFile@6522} 从DefaultMultipartHttpServletRequest.multipartFiles中获取
```

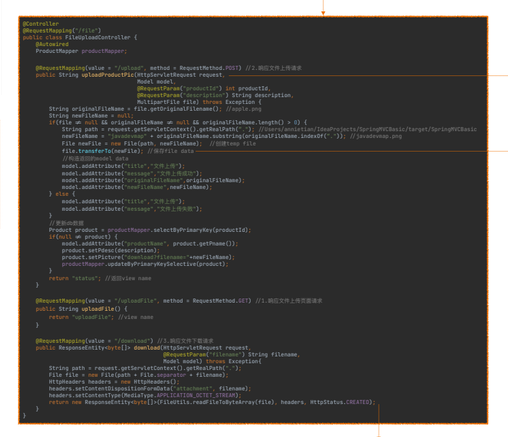
```
//CASE1: 处理uploadProductPic()返回值
void ViewNameMethodReturnValueHandler.handleReturnValue(Object returnValue, MethodParameter returnType,
                                                         ModelAndViewContainer mavContainer, NativeWebRequest webRequest) {
  String viewName = returnValue.toString();
  /* 保存返回的视图名称
  mavContainer.setViewName(viewName);
  if (this.isRedirectViewName(viewName)) mavContainer.setRedirectModelScenario(true);
}
```



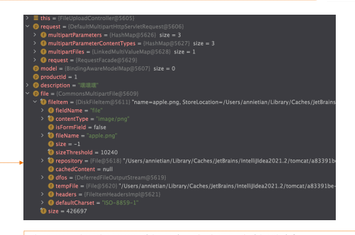
```
v mavContainer = {ModelAndViewContainer@5605} "ModelAndViewContainer: reference to view wi
    ignoreDefaultModelOnRedirect = false
  > view = "status"   FileUploadController.uploadProductPic()返回的view name
  v defaultModel = {BindingAwareModelMap@5628}  size = 5
    > "title" -> "文件上传"
    > "message" -> "文件上传成功"
    > "originalFileName" -> "apple.png"
    > "newFileName" -> "javadevmap.png"
    > "productName" -> "apple"
    redirectModel = null
    redirectModelScenario = false
    bindingDisabledAttributes = {HashSet@5629}  size = 0
    status = null
```

1. 作为参数传给FileUploadController.uploadProductPic()
2. 处理请求过程中赋值

```
//CASE2: 处理download()的返回值
void HttpEntityMethodProcessor.handleReturnValue(Object returnValue, MethodParameter returnType,
                                                  ModelAndViewContainer mavContainer, NativeWebRequest webRequest) {
  mavContainer.setRequestHandled(true);
  if (returnValue != null) {
    ServletServerHttpRequest inputMessage = this.createInputMessage(webRequest);
    ServletServerHttpResponse outputMessage = this.createOutputMessage(webRequest);
    HttpEntity<?> responseEntity = (HttpEntity)returnValue;
    …
    //将responseEntity中的文件内容写入webRequest.response.outputStream.byteBuffer
    this.writeWithMessageConverters(responseEntity.getBody(), returnType, inputMessage, outputMessage);
    outputMessage.flush();
  }
}
```

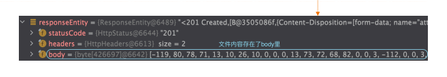


```
@Controller
@RequestMapping("/file")
public class FileUploadController {
    @Autowired
    ProductMapper productMapper;

    @RequestMapping(value = "/upload", method = RequestMethod.POST) //2.响应文件上传请求
    public String uploadProductPic(HttpServletRequest request,
                                   Model model,
                                   @RequestParam("productId") int productId,
                                   @RequestParam("description") String description,
                                   MultipartFile file) throws Exception {
        String originalFileName = file.getOriginalFilename(); //apple.png
        String newFileName = null;
        if(file != null && originalFileName != null && originalFileName.length() > 0) {
            String path = request.getServletContext().getRealPath("."); //Users/annietian/IdeaProjects/SpringMVCBasic/target/SpringMVCBasic
            newFileName = "javadevmap" + originalFileName.substring(originalFileName.indexOf(".")); //javadevmap.png
            File newFile = new File(path, newFileName);  //创建temp file
            file.transferTo(newFile); //保存file data
            //构造返回的model data
            model.addAttribute("title","文件上传");
            model.addAttribute("message","文件上传成功");
            model.addAttribute("originalFileName",originalFileName);
            model.addAttribute("newFileName",newFileName);
        } else {
            model.addAttribute("title","文件上传");
            model.addAttribute("message","文件上传失败");
        }
        //更新db数据
        Product product = productMapper.selectByPrimaryKey(productId);
        if(null != product) {
            model.addAttribute("productName", product.getPname());
            product.setPdesc(description);
            product.setPicture("download?filename="+newFileName);
            productMapper.updateByPrimaryKeySelective(product);
        }
        return "status"; //返回view name
    }

    @RequestMapping(value = "/uploadFile", method = RequestMethod.GET) //1.响应文件上传页面请求
    public String uploadFile() {
        return "uploadFile"; //view name
    }

    @RequestMapping(value = "/download") //3.响应文件下载请求
    public ResponseEntity<byte[]> download(HttpServletRequest request,
                                           @RequestParam("filename") String filename,
                                           Model model) throws Exception{
        String path = request.getServletContext().getRealPath(".");
        File file = new File(path + File.separator + filename);
        HttpHeaders headers = new HttpHeaders();
        headers.setContentDispositionFormData("attachment", filename);
        headers.setContentType(MediaType.APPLICATION_OCTET_STREAM);
        return new ResponseEntity<byte[]>(FileUtils.readFileToByteArray(file), headers, HttpStatus.CREATED);
    }
}
```



```
> this = {FileUploadController@5605}
v request = {DefaultMultipartHttpServletRequest@5606}
  > multipartParameters = {HashMap@5626}  size = 3
  > multipartParameterContentTypes = {HashMap@5627}  size = 3
  > multipartFiles = {LinkedMultiValueMap@5628}  size = 1
  > request = {RequestFacade@5629}
> model = {BindingAwareModelMap@5607}  size = 0
  productId = 1
> description = "嘿嘿嘿"
v file = {CommonsMultipartFile@5609}
  v fileItem = {DiskFileItem@5611} "name=apple.png, StoreLocation=/Users/annietian/Library/Caches/JetBrains
    > fieldName = "file"
    > contentType = "image/png"
      isFormField = false
    > fileName = "apple.png"
      size = -1
      sizeThreshold = 10240
    > repository = {File@5618} "/Users/annietian/Library/Caches/JetBrains/IntelliJIdea2021.2/tomcat/a83391b
      cachedContent = null
    > dfos = {DeferredFileOutputStream@5619}
    > tempFile = {File@5620} "/Users/annietian/Library/Caches/JetBrains/IntelliJIdea2021.2/tomcat/a83391be-
    > headers = {FileItemHeadersImpl@5621}
    > defaultCharset = "ISO-8859-1"
    size = 426697
```

```
void CommonsMultipartFile.transferTo(File dest) → DiskFileItem.write(File file) {
  if (this.isInMemory()) { //CASE1: 文件存在内存中
    FileOutputStream fout = null;
    try {
      fout = new FileOutputStream(file);
      fout.write(this.get()); //将ByteArrayOutputStream.buffers中的数据汇总到byte[], 写入file
      fout.close();
    } finally { IOUtils.closeQuietly(fout); }
  }
  else { //CASE2: 文件存在磁盘
    File outputFile = this.getStoreLocation();
    this.size = outputFile.length();
    FileUtils.moveFile(outputFile, file); //先尝试rename暂存磁盘的临时文件, 如果失败, 使用FileChannel将数据copy到file
  }
}
```



```
v responseEntity = {ResponseEntity@6489} "<201 Created,[B@3505086f,[Content-Disposition=[form-data; name="at
  > statusCode = {HttpStatus@6644} "201"
  > headers = {HttpHeaders@6613}  size = 2      文件内容存在了body里
  > body = {byte[426697]@6642} [-119, 80, 78, 71, 13, 10, 26, 10, 0, 0, 0, 13, 73, 72, 68, 82, 0, 0, 3, -112, 0, 0, 3,
```

```
void CommonsMultipartFile.transferTo(File dest) → DiskFileItem.write(File file) {
  if (this.isInMemory()) { //CASE1: 文件存在内存中
    FileOutputStream fout = null;
    try {
      fout = new FileOutputStream(file);
      fout.write(this.get()); //将ByteArrayOutputStream.buffers中的数据汇总到byte[], 写入file
      fout.close();
    } finally { IOUtils.closeQuietly(fout); }
  }
  else { //CASE2: 文件存在磁盘
    File outputFile = this.getStoreLocation();
    this.size = outputFile.length();
    FileUtils.moveFile(outputFile, file); //先尝试rename暂存磁盘的临时文件, 如果失败, 使用FileChannel将数据copy到file
  }
}
```